

Leaf Health Check - Main Streamlit Application v2.0

AI-powered plant leaf disease detection, severity assessment, and diagnosis system.

Features:

- 7-tab UI: Analyze, Dashboard, History, AI Assistant, Care Plan, Encyclopedia, About
- Real-time discoloration heatmap overlay
- Interactive bar & pie charts for analysis results
- Session-based history with statistics dashboard
- Health score calculation (0-100)
- Disease encyclopedia with 8 detailed entries
- Gemini AI integration (graceful fallback without API key)
- JSON & CSV export of results and history
- Side-by-side original vs heatmap comparison
- Fully responsive layout with custom CSS theming

Deploy:

```
streamlit run app.py
```

```
"""
```

```
import streamlit as st
import numpy as np
import cv2
import json
import sys
import logging
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from pathlib import Path
from datetime import datetime
from dotenv import load_dotenv
```

```
# — Environment & path setup
```

```
load_dotenv()
sys.path.insert(0, str(Path(__file__).parent))

from utils.preprocess import ImagePreprocessor
from utils.severity import SeverityGrader
from utils.recommendations import RecommendationEngine
from utils.gemini_ai import get_gemini_engine
from model.train import PlantDiseaseModel
from database.init_db import init_database, get_connection
```

— Logging

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)
logger = logging.getLogger(__name__)
```

— Page config

```
st.set_page_config(
    page_title="🌿 Leaf Health Check",
    page_icon="🌿",
    layout="wide",
    initial_sidebar_state="expanded",
    menu_items={
        "Get Help": "https://github.com/leaf-health-check",
        "Report a bug": "mailto:support@leafhealthcheck.com",
        "About": "Leaf Health Check v2.0 — AI-Powered Plant Disease Detection",
    },
)
```

— Custom CSS

```
st.markdown("""
<style>
.main { background-color: #f0f8f5; }

.stButton > button {
    width: 100%;
    background: linear-gradient(135deg, #2ecc71, #27ae60);
    color: white; border: none; border-radius: 8px;
    padding: 0.6rem 1.2rem; font-weight: 600;
    transition: transform 0.15s ease, box-shadow 0.15s ease;
}
.stButton > button:hover {
    transform: translateY(-2px);
    box-shadow: 0 6px 16px rgba(46,204,113,0.4);
}

.disease-card {
    background: white; padding: 18px 22px; border-radius: 12px;
    margin: 8px 0; box-shadow: 0 2px 8px rgba(0,0,0,0.08);
    color: #333; border-left: 4px solid #2ecc71;
}

```

```

.warning-card {
  background: #fff9e6; padding: 18px 22px; border-radius: 12px;
  margin: 8px 0; box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  color: #333; border-left: 4px solid #f39c12;
}

.danger-card {
  background: #fef0f0; padding: 18px 22px; border-radius: 12px;
  margin: 8px 0; box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  color: #333; border-left: 4px solid #e74c3c;
}

.info-card {
  background: #f0f8ff; padding: 18px 22px; border-radius: 12px;
  margin: 8px 0; box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  color: #333; border-left: 4px solid #3498db;
}

.severity-badge {
  padding: 8px 18px; border-radius: 20px; color: white;
  font-weight: 700; display: inline-block; font-size: 0.9rem;
  letter-spacing: 0.5px;
}

.metric-card {
  text-align: center; padding: 18px 12px; background: white;
  border-radius: 12px; box-shadow: 0 2px 8px rgba(0,0,0,0.07);
}

.metric-value { font-size: 1.9rem; font-weight: 700; color: #2ecc71; margin: 4px 0; }
.metric-label { font-size: 0.78rem; color: #888; text-transform: uppercase; letter-spacing: 0.8px; }

.health-score { font-size: 3.2rem; font-weight: 900; text-align: center; margin: 8px 0; }

.header-banner {
  background: linear-gradient(135deg, #1a5c2a, #2ecc71);
  padding: 26px 32px; border-radius: 16px; color: white;
  margin-bottom: 20px; text-align: center;
}

.header-banner h1 { color: white; font-size: 2.4rem; margin: 0; }
.header-banner p { color: rgba(255,255,255,0.85); margin: 6px 0 0 0; font-size: 1.05rem; }

.tip-box {
  background: #f1fdf5; border: 1px solid #b7e4c7;
  padding: 12px 16px; border-radius: 10px; margin: 5px 0; color: #1e4d2b;
}

.footer {
  text-align: center; color: #aaa; font-size: 0.82rem;
  padding: 16px 0 4px 0; border-top: 1px solid #e0e0e0; margin-top: 40px;
}

```

```
}  
</style>  
"""', unsafe_allow_html=True)
```

— Constants

```
APP_VERSION      = "2.0.0"  
SUPPORTED_PLANTS = ["Tomato", "Potato", "Apple", "Corn", "Wheat"]  
SUPPORTED_DISEASES = [  
    "Early Blight", "Late Blight", "Septoria Leaf Spot",  
    "Powdery Mildew", "Rust", "Gray Leaf Spot", "Leaf Scab", "Healthy",  
]  
SEVERITY_LEVELS = ["Healthy", "Mild", "Moderate", "Severe", "Dying"]  
CLIMATE_ZONES   = ["Tropical", "Subtropical", "Temperate", "Semi-Arid", "Cold"]
```

— Session state defaults

```
for key, val in {  
    "analysis_history": [],  
    "model": None,  
    "chat_messages": [],  
    "total_analyses": 0,  
    "session_start": datetime.now().isoformat(),  
    "last_analysis": None,  
}.items():  
    if key not in st.session_state:  
        st.session_state[key] = val  
  
if "gemini_engine" not in st.session_state:  
    st.session_state.gemini_engine = get_gemini_engine()
```

— Cached resource loaders

```
@st.cache_resource(show_spinner="🔄 Loading AI models...")  
def load_model():  
    """Load or build CNN models — cached across reruns."""  
    try:  
        model = PlantDiseaseModel(architecture="efficientnet")  
        model_path = Path("model")  
        d_ok = model.load_model("disease", str(model_path))  
        p_ok = model.load_model("plant", str(model_path))  
        if not d_ok:  
            logger.info("Building disease detection model...")  
            model.build_disease_model()  
        if not p_ok:  
            logger.info("Building plant species model...")  
            model.build_plant_model()
```

```

    return model
except Exception as exc:
    logger.error(f"Model load error: {exc}")
    return None

```

@st.cache_resource(show_spinner="🔄 Initialising database...")

```

def init_db():
    """Ensure the database exists and return its path string."""
    try:
        db_path = Path("database/plants.db")
        db_path.parent.mkdir(parents=True, exist_ok=True)
        if not db_path.exists():
            init_database()
        return str(db_path)
    except Exception as exc:
        logger.error(f"DB init error: {exc}")
        return None

```

— Utility functions

```

def compute_health_score(severity: str, affected_pct: float, confidence: float) -> int:
    """Return an integer 0-100 health score. 100 = perfectly healthy."""
    base = {"healthy": 100, "mild": 75, "moderate": 50, "severe": 25, "dying": 5}.get(severity, 50)
    penalty = min(affected_pct * 0.3, 20)
    bonus = (confidence - 0.5) * 10 if confidence > 0.5 else 0
    return int(max(0, min(100, base - penalty + bonus)))

```

```

def health_score_color(score: int) -> str:
    if score >= 80: return "#2ecc71"
    if score >= 60: return "#f39c12"
    if score >= 40: return "#e67e22"
    if score >= 20: return "#e74c3c"
    return "#7f0000"

```

```

def build_heatmap_image(original_rgb: np.ndarray, discoloration_data: dict) -> np.ndarray:
    """Overlay coloured masks on original image to show diseased regions."""
    overlay = original_rgb.copy().astype(np.uint8)
    masks = discoloration_data.get("masks", {})
    cmap = {"yellow": (255, 230, 50), "brown": (160, 82, 45),
            "black": (30, 30, 30), "white": (220, 220, 220)}
    for name, rgb in cmap.items():
        mask = masks.get(name)
        if mask is not None and np.any(mask):

```

```

coloured = np.zeros_like(overlay)
coloured[:] = rgb
overlay = np.where(
    mask[:, :, np.newaxis] > 0,
    cv2.addWeighted(overlay, 0.45, coloured, 0.55, 0),
    overlay,
)
return overlay.astype(np.uint8)

```

```

def make_discoloration_bar(cb: dict) -> plt.Figure:
    labels = ["● Black", "● Brown", "● Yellow", "● White"]
    values = [cb.get("black_pixels", 0), cb.get("brown_pixels", 0),
              cb.get("yellow_pixels", 0), cb.get("white_pixels", 0)]
    colours = ["#2c2c2c", "#8B4513", "#FFD700", "#D3D3D3"]
    total = sum(values) or 1
    fig, ax = plt.subplots(figsize=(5, 2.6))
    fig.patch.set_facecolor("#f0f8f5"); ax.set_facecolor("#f0f8f5")
    bars = ax.barh(labels, [v / total * 100 for v in values], color=colours, height=0.5)
    for bar in bars:
        ax.text(bar.get_width() + 0.5, bar.get_y() + bar.get_height() / 2,
                f"{bar.get_width():.1f}%", va="center", fontsize=9, color="#555")
    ax.set_xlabel("% of affected pixels", fontsize=9); ax.set_xlim(0, 115)
    ax.spines[["top", "right"]].set_visible(False); ax.tick_params(labelsize=9)
    plt.tight_layout(); return fig

```

```

def make_confidence_pie(dc: float, pc: float, oc: float) -> plt.Figure:
    labels = ["Disease\nDetection", "Plant\nSpecies", "Overall\nDiagnosis"]
    values = [dc * 100, pc * 100, oc * 100]
    fig, ax = plt.subplots(figsize=(4, 3))
    fig.patch.set_facecolor("#f0f8f5")
    wedges, _, autotexts = ax.pie(
        values, labels=labels, colors=["#2ecc71", "#3498db", "#9b59b6"],
        autopct="%1.1f%%", startangle=140,
        wedgeprops={"linewidth": 1.5, "edgecolor": "white"},
        textprops={"fontsize": 8},
    )
    for at in autotexts:
        at.set_color("white"); at.set_fontweight("bold")
    ax.set_title("Confidence Breakdown", fontsize=10, pad=8)
    plt.tight_layout(); return fig

```

```

def make_history_trend(history: list) -> plt.Figure | None:
    if len(history) < 2:
        return None

```

```

    scores = [compute_health_score(h["severity"], h["affected_percentage"],
h["diagnosis_confidence"])
    for h in history]
    idx = list(range(1, len(scores) + 1))
    fig, ax = plt.subplots(figsize=(7, 2.8))
    fig.patch.set_facecolor("#f0f8f5"); ax.set_facecolor("#f0f8f5")
    ax.plot(idx, scores, marker="o", color="#2ecc71", linewidth=2.5,
            markersize=7, markerfacecolor="white", markeredgewidth=2)
    ax.fill_between(idx, scores, alpha=0.12, color="#2ecc71")
    ax.axhline(70, color="#f39c12", linestyle="--", linewidth=1, alpha=0.6, label="Mild
threshold")
    ax.axhline(40, color="#e74c3c", linestyle="--", linewidth=1, alpha=0.6, label="Severe
threshold")
    ax.set_xlabel("Analysis #", fontsize=9); ax.set_ylabel("Health Score", fontsize=9)
    ax.set_ylim(0, 105); ax.legend(fontsize=8, framealpha=0.4)
    ax.spines[["top", "right"]].set_visible(False); ax.tick_params(labelsize=9)
    plt.tight_layout(); return fig

```

```

def make_disease_distribution_chart(history: list) -> plt.Figure:
    disease_counts: dict[str, int] = {}
    for h in history:
        d = h["disease"].replace("_", " ").title()
        disease_counts[d] = disease_counts.get(d, 0) + 1
    fig, ax = plt.subplots(figsize=(6, 2.8))
    fig.patch.set_facecolor("#f0f8f5"); ax.set_facecolor("#f0f8f5")
    colours = plt.cm.Set2(np.linspace(0, 1, len(disease_counts)))
    ax.barh(list(disease_counts.keys()), list(disease_counts.values()),
            color=colours, height=0.5)
    ax.set_xlabel("Count", fontsize=9)
    ax.spines[["top", "right"]].set_visible(False); ax.tick_params(labelsize=9)
    plt.tight_layout(); return fig

```

— UI helpers

```

def render_severity_badge(severity: str):
    badge = SeverityGrader.get_severity_badge(severity)
    st.markdown(
        f'<span class="severity-badge" style="background:{badge["color"]}">'
        f'{badge["emoji"]} {badge["display"].upper()}</span>',
        unsafe_allow_html=True,
    )
    st.caption(badge["description"])

```

```

def render_health_score(score: int):

```

```

colour = health_score_color(score)
st.markdown(
    f'<div style="text-align:center">'
    f'<div class="health-score" style="color:{colour}">{score}</div>'
    f'<div style="color:#888;font-size:0.85rem">Health Score / 100</div>'
    f'</div>',
    unsafe_allow_html=True,
)

```

```

def render_metric_card(label: str, value: str, sub: str = ""):
    st.markdown(
        f'<div class="metric-card">'
        f'<div class="metric-label">{label}</div>'
        f'<div class="metric-value">{value}</div>'
        f'<div style="color:#aaa;font-size:0.78rem">{sub}</div>'
        f'</div>',
        unsafe_allow_html=True,
    )

```

— Core analysis pipeline

```

def analyze_leaf_image(image_path: str) -> dict | None:
    """
    Full analysis pipeline:
    Load → Preprocess → Discoloration → CNN Disease → CNN Plant →
    Severity → Recommendations → Health Score → Heatmap → Return dict
    """
    try:
        # 1. Load
        image = ImagePreprocessor.load_image(image_path)
        original_image = image.copy()

        # 2. Discoloration (pixel-level HSV analysis)
        discoloration_data = ImagePreprocessor.detect_discoloration(image)

        # 3. Heatmap overlay for visualisation
        heatmap_image = build_heatmap_image(original_image, discoloration_data)

        # 4. Preprocess for CNN input
        processed = ImagePreprocessor.preprocess_for_model(image)

        # 5. Load CNN models and predict
        model = load_model()
        if model is None:
            st.error("❌ Model unavailable. Run `python model/create_models.py` first.")
            return None
    
```



```
disease_result = model.predict_disease(processed)
plant_result = model.predict_plant(processed)
```

```
# 6. Severity grading
```

```
severity_result = SeverityGrader.calculate_severity(
    discoloration_data,
    disease_result["disease"],
    disease_result["confidence"],
)
```

```
# 7. Recommendations from DB + fallback
```

```
db_path = init_db()
recommendations = RecommendationEngine.get_recommendations(
    disease_result["disease"],
    severity_result["severity_level"],
    plant_result["plant"],
    db_path,
)
```

```
# 8. Health score (0–100 composite metric)
```

```
health_score = compute_health_score(
    severity_result["severity_level"],
    severity_result["affected_percentage"],
    severity_result["diagnosis_confidence"],
)
```

```
# 9. Assemble full result dictionary
```

```
analysis = {
    "plant": plant_result["plant"],
    "plant_confidence": plant_result["confidence"],
    "plant_all_predictions": plant_result.get("predictions", {}),
    "disease": disease_result["disease"],
    "disease_confidence": disease_result["confidence"],
    "disease_all_predictions": disease_result.get("predictions", {}),
    "severity": severity_result["severity_level"],
    "affected_percentage": severity_result["affected_percentage"],
    "weighted_score": severity_result["weighted_score"],
    "diagnosis_confidence": severity_result["diagnosis_confidence"],
    "discoloration_breakdown": severity_result["color_breakdown"],
    "recommendations": recommendations,
    "health_score": health_score,
    "original_image": original_image,
    "heatmap_image": heatmap_image,
    "timestamp": datetime.now().isoformat(),
    "image_path": image_path,
}
```

```

# 10. Persist in session state
st.session_state.analysis_history.append(analysis)
st.session_state.total_analyses += 1
st.session_state.last_analysis = analysis

# 11. Persist in SQLite history table
if db_path:
    RecommendationEngine.save_analysis_history(
        {
            "plant_name":      analysis["plant"],
            "disease_name":    analysis["disease"],
            "severity":        analysis["severity"],
            "confidence":      analysis["diagnosis_confidence"],
            "discoloration_percent": analysis["affected_percentage"],
            "image_filename":   Path(image_path).name,
        },
        db_path,
    )

return analysis

except Exception as exc:
    logger.error(f"Analysis pipeline error: {exc}", exc_info=True)
    st.error(f"❌ Analysis failed: {exc}")
    return None

```

#

TAB FUNCTIONS

#

— Tab 1: Analyze

```

def tab_analyze():
    st.markdown("## 🔍 Upload & Analyze Leaf")
    st.markdown("Upload a clear, well-lit photo of a single leaf for best accuracy.")

    up_col, res_col = st.columns([1, 1], gap="large")

    with up_col:
        st.markdown("### 📷 Image Upload")
        uploaded = st.file_uploader(
            "Choose a leaf image (JPG / PNG)",

```

)

if

ir

u

fr

if

W

W

if

S

)

```

st.markdown("### 🧪 Run Analysis")
if st.button("🚀 Analyze Leaf", use_container_width=True):
    with st.spinner("🧠 Running AI analysis pipeline..."):
        analysis = analyze_leaf_image("temp_image.jpg")
    if not analysis:
        return

# — Results Section


---



st.markdown("---")
st.markdown("## 📋 Diagnosis Results")

# Key metrics row
c1, c2, c3, c4, c5 = st.columns(5)
badge = SeverityGrader.get_severity_badge(analysis["severity"])
with c1: render_metric_card("Plant",
    analysis["plant"].title(),
    f'{analysis["plant_confidence"]:.0%} conf')
with c2: render_metric_card("Disease",
    analysis["disease"].replace("_", " ").title(),
    f'{analysis["disease_confidence"]:.0%} conf')
with c3: render_metric_card("Affected Area",
    f'{analysis["affected_percentage"]:.1f}%', "of leaf")
with c4: render_metric_card("Severity",
    f'{badge["emoji"]} {badge["display"]}',
    badge["description"])
with c5: render_health_score(analysis["health_score"])

st.markdown("---")

# Original vs Heatmap
st.markdown("### 🖼️ Original vs Discoloration Heatmap")
ic1, ic2 = st.columns(2)
with ic1: st.image(analysis["original_image"], caption="Original Leaf",
    use_container_width=True)
with ic2: st.image(analysis["heatmap_image"], caption="Discoloration Heatmap",
    use_container_width=True)

st.markdown("---")

# Charts
st.markdown("### 📊 Analysis Charts")
ch1, ch2 = st.columns(2)
with ch1:
    st.markdown("***Discoloration Breakdown**")
    st.pyplot(make_discoloration_bar(analysis["discoloration_breakdown"]),
    use_container_width=True)
with ch2:

```

```

st.markdown("***Confidence Breakdown**")
st.pyplot(make_confidence_pie(
    analysis["disease_confidence"],
    analysis["plant_confidence"],
    analysis["diagnosis_confidence"],
), use_container_width=True)

st.markdown("---")

# Rescue recommendations
st.markdown("### 🆘 Rescue Recommendations")
card_styles = ["disease-card", "warning-card", "danger-card"]
for i, tip in enumerate(analysis["recommendations"][:3], 1):
    style = card_styles[min(i - 1, 2)]
    st.markdown(f"<div class='{style}'><b>💡 Tip {i}:</b> {tip}</div>",
        unsafe_allow_html=True)

# Gemini AI enhancements
ge = st.session_state.gemini_engine
if ge and ge._initialized:
    st.markdown("---")
    st.markdown("### 🤖 AI-Enhanced Insights (Gemini)")
    with st.spinner("✨ Generating AI insights..."):
        explanation = ge.generate_disease_explanation(
            analysis["disease"], analysis["plant"],
            analysis["severity"], analysis["affected_percentage"],
        )
    st.info(f"📖 **Disease Overview:** {explanation}")

    tips_result = ge.generate_personalized_tips(
        analysis["disease"], analysis["plant"],
        analysis["severity"], analysis["affected_percentage"],
        analysis["recommendations"],
    )
    if tips_result["status"] == "success":
        st.success(f"🎯 **AI Tips:**\n{tips_result['enhanced_tips']}")

    preventive = ge.identify_preventive_measures(
        analysis["plant"], analysis["disease"]
    )
    if preventive:
        st.markdown("*** 🛡️ Preventive Measures:***")
        for m in preventive:
            st.markdown(f"&nbsp;&nbsp;&nbsp;✅ {m}")

# Probability breakdowns
with st.expander("🔬 All Disease Probabilities"):
    probs = analysis.get("disease_all_predictions", {})

```

```

for cls, prob in sorted(probs.items(), key=lambda x: x[1], reverse=True):
    st.progress(float(prob), text=f"{cls.replace('_', ' ').title(): {prob:.1%}}")

with st.expander("🌿 All Plant Species Probabilities"):
    pprobs = analysis.get("plant_all_predictions", {})
    for cls, prob in sorted(pprobs.items(), key=lambda x: x[1], reverse=True):
        st.progress(float(prob), text=f"{cls.title(): {prob:.1%}}")

# Export
st.markdown("---")
st.markdown("### 📄 Export Results")
export_dict = {
    "plant": analysis["plant"], "plant_confidence": analysis["plant_confidence"],
    "disease": analysis["disease"], "disease_confidence":
analysis["disease_confidence"],
    "severity": analysis["severity"], "health_score": analysis["health_score"],
    "affected_percentage": analysis["affected_percentage"],
    "diagnosis_confidence": analysis["diagnosis_confidence"],
    "discoloration": analysis["discoloration_breakdown"],
    "recommendations": analysis["recommendations"],
    "timestamp": analysis["timestamp"], "app_version": APP_VERSION,
}
ec1, ec2 = st.columns(2)
with ec1:
    st.download_button("📄 Download JSON",
        data=json.dumps(export_dict, indent=2),

file_name=f"leaf_analysis_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json",
    mime="application/json", use_container_width=True)
with ec2:
    st.download_button("📊 Download CSV",
        data=pd.DataFrame([export_dict]).to_csv(index=False),

file_name=f"leaf_analysis_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv",
    mime="text/csv", use_container_width=True)

```

— Tab 2: Dashboard

```

def tab_dashboard():
    st.markdown("### 📊 Session Dashboard")
    history = st.session_state.analysis_history
    if not history:
        st.info("📁 No analyses yet. Go to **🔍 Analyze Leaf** to get started.")
        return

    total = len(history)

```

```

avg_score = sum(h["health_score"] for h in history) / total
avg_aff = sum(h["affected_percentage"] for h in history) / total
worst = min(history, key=lambda h: h["health_score"])
best = max(history, key=lambda h: h["health_score"])
most_common_disease = max(
    set(h["disease"] for h in history),
    key=lambda d: sum(1 for h in history if h["disease"] == d)
).replace("_", " ").title()

m1, m2, m3, m4 = st.columns(4)
with m1: render_metric_card("Total Analyses", str(total), "this session")
with m2: render_metric_card("Avg Health Score", f"{avg_score:.0f}/100", "higher is better")
with m3: render_metric_card("Avg Affected Area", f"{avg_aff:.1f}%", "of leaf surface")
with m4: render_metric_card("Most Common Disease", most_common_disease, "by
frequency")

st.markdown("---")
st.markdown("### 📈 Health Score Trend")
trend = make_history_trend(history)
if trend:
    st.pyplot(trend, use_container_width=True)
else:
    st.info("Analyse at least 2 leaves to see the trend.")

st.markdown("---")
st.markdown("### 🦠 Disease Distribution")
st.pyplot(make_disease_distribution_chart(history), use_container_width=True)

st.markdown("---")
bc, wc = st.columns(2)
with bc:
    st.markdown("### 🏆 Best Result")
    st.markdown(
        f'<div class="disease-card"><b>Plant:</b> {best["plant"].title()}<br>'
        f'<b>Disease:</b> {best["disease"].replace("_", " ").title()}<br>'
        f'<b>Health Score:</b> {best["health_score"]}/100<br>'
        f'<b>Severity:</b> {best["severity"].title()}<br>'
        f'<b>Time:</b> {best["timestamp"][:19]}</div>',
        unsafe_allow_html=True,
    )
with wc:
    st.markdown("### ⚠️ Worst Result")
    st.markdown(
        f'<div class="danger-card"><b>Plant:</b> {worst["plant"].title()}<br>'
        f'<b>Disease:</b> {worst["disease"].replace("_", " ").title()}<br>'
        f'<b>Health Score:</b> {worst["health_score"]}/100<br>'
        f'<b>Severity:</b> {worst["severity"].title()}<br>'
        f'<b>Time:</b> {worst["timestamp"][:19]}</div>',

```

```

        unsafe_allow_html=True,
    )

```

— Tab 3: History

```

def tab_history():
    st.markdown("## 📊 Analysis History")
    history = st.session_state.analysis_history
    if not history:
        st.info("📁 No history yet. Analyse a leaf first.")
        return

    st.markdown(f"Showing **{len(history)}** analyses from this session.")

    fc1, fc2, fc3 = st.columns(3)
    with fc1: filter_plant = st.selectbox("Filter by Plant", ["All"] + SUPPORTED_PLANTS)
    with fc2: filter_sev = st.selectbox("Filter by Severity", ["All"] + [s.lower() for s in
SEVERITY_LEVELS])
    with fc3: sort_by = st.selectbox("Sort by", ["Newest First", "Oldest First", "Worst
Health", "Best Health"])

    filtered = list(history)
    if filter_plant != "All":
        filtered = [h for h in filtered if h["plant"].lower() == filter_plant.lower()]
    if filter_sev != "All":
        filtered = [h for h in filtered if h["severity"].lower() == filter_sev.lower()]
    if sort_by == "Newest First":
        filtered = list(reversed(filtered))
    elif sort_by == "Worst Health":
        filtered = sorted(filtered, key=lambda h: h["health_score"])
    elif sort_by == "Best Health":
        filtered = sorted(filtered, key=lambda h: h["health_score"], reverse=True)

    if not filtered:
        st.warning("No results match the selected filters."); return

    table_data = []
    for i, h in enumerate(filtered, 1):
        badge = SeverityGrader.get_severity_badge(h["severity"])
        table_data.append({
            "#": i, "Plant": h["plant"].title(),
            "Disease": h["disease"].replace("_", " ").title(),
            "Severity": f'{badge["emoji"]} {h["severity"].title()}',
            "Health Score": h["health_score"],
            "Affected %": f'{h["affected_percentage"]:.1f}%',
            "Confidence": f'{h["diagnosis_confidence"]:.0%}',

```



```

        "Time": h["timestamp"][:19],
    })
df = pd.DataFrame(table_data)
st.dataframe(df, use_container_width=True, hide_index=True)
st.download_button("📄 Export History (CSV)",
    data=df.to_csv(index=False),
    file_name=f"leaf_history_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv",
    mime="text/csv")

st.markdown("---")
st.markdown("### 📇 Detailed Cards")
for i, h in enumerate(filtered, 1):
    badge = SeverityGrader.get_severity_badge(h["severity"])
    scls = {"healthy": "disease-card", "mild": "disease-card",
"moderate": "warning-card", "severe": "danger-card", "dying": "danger-card"}.get(h["severity"], "info-card")
    with st.expander(
        f#{i} — {h["plant"].title()} · {h["disease"].replace("_", " ").title()} '
        f· {badge["emoji"]} {h["severity"].title()} — {h["timestamp"][:10]}
    ):
        dc1, dc2, dc3 = st.columns(3)
        with dc1: st.metric("Health Score", f{h["health_score"]}/100')
        with dc2: st.metric("Affected Area", f{h["affected_percentage"]:.1f}%)
        with dc3: st.metric("Confidence", f{h["diagnosis_confidence"]:.0f}%)
        st.markdown("***Recommendations:***")
        for tip in h["recommendations"]:
            st.markdown(f<div class="tip-box">✅ {tip}</div>', unsafe_allow_html=True)

```

— Tab 4: AI Assistant

```

def tab_ai_assistant():
    st.markdown("### 🤖 AI Plant Health Assistant")
    st.markdown("Ask anything about plant diseases, treatments, soil, or watering.")

    ge = st.session_state.gemini_engine
    if not (ge and ge._initialized):
        st.warning(
            "⚠️ AI Assistant requires a **Google Gemini API key**.\n\n"
            "1. Get a **free key** at https://aistudio.google.com/app/apikey\n"
            "2. Open `.env` → set `GOOGLE_GEMINI_API_KEY=your_key`\n"
            "3. Restart: `streamlit run app.py`"
        )
    return

st.markdown("***💬 Quick Questions:***")
qq_cols = st.columns(4)

```

```

quick_qs = [
    "How do I treat Early Blight?",
    "What causes Powdery Mildew?",
    "How often to water tomatoes?",
    "Best fungicide for Late Blight?",
]
for col, q in zip(qq_cols, quick_qs):
    with col:
        if st.button(q, key=f"qq_{q}"):
            st.session_state.chat_messages.append({"role": "user", "content": q})
            with st.spinner("🤖 Thinking..."):
                resp = ge.chat(q)
            st.session_state.chat_messages.append({"role": "assistant", "content": resp})
            st.rerun()

st.markdown("---")
st.markdown("### 💬 Conversation")

if not st.session_state.chat_messages:
    st.markdown(
        '<div class="info-card">👋 Start a conversation by typing below or '
        'clicking a quick question above.</div>',
        unsafe_allow_html=True,
    )

for msg in st.session_state.chat_messages:
    if msg["role"] == "user":
        st.markdown(f'<div class="disease-card">👤 <b>You:</b> {msg["content"]}</div>',
                    unsafe_allow_html=True)
    else:
        st.markdown(f'<div class="info-card">🤖 <b>AI:</b> {msg["content"]}</div>',
                    unsafe_allow_html=True)

inp1, inp2, inp3 = st.columns([5, 1, 1])
with inp1:
    user_input = st.text_input("Type your question...", key="chat_input",
                               placeholder="e.g. How do I prevent rust on wheat?",
                               label_visibility="collapsed")
with inp2:
    send = st.button("Send 📤", use_container_width=True)
with inp3:
    clear = st.button("Clear 🗑️", use_container_width=True)

if send and user_input:
    st.session_state.chat_messages.append({"role": "user", "content": user_input})
    with st.spinner("🤖 Thinking..."):
        resp = ge.chat(user_input)
    st.session_state.chat_messages.append({"role": "assistant", "content": resp})

```

```

st.rerun()

if clear:
    st.session_state.chat_messages = []; ge.clear_history()
    st.success("Chat cleared!"); st.rerun()

if st.session_state.chat_messages:
    chat_text = "\n\n".join(f'[{m["role"].upper()}] {m["content"]}'
                             for m in st.session_state.chat_messages)
    st.download_button(
        "

```

— Tab 5: Care Plan

```

def tab_care_plan():
    st.markdown("##  Disease", SUPPORTED_DISEASES)
    with cp2:
        sev_sel = st.selectbox(" Severity Level", SEVERITY_LEVELS[1:])
        climate_sel = st.selectbox(" Climate Zone", CLIMATE_ZONES)

    extra = st.text_area(
        " Additional Notes (optional)",
        placeholder="e.g. greenhouse, organic only, last treated 2 weeks ago...",
        height=80)

    if st.button(" Generate Care Plan", use_container_width=True):
        if ge and ge._initialized:
            with st.spinner(" Crafting personalised care plan..."):
                care_plan = ge.generate_care_plan(plant_sel, disease_sel, sev_sel)
            else:
                care_plan = (
                    f"**{plant_sel} — {disease_sel} ({sev_sel} Severity) | Climate: {climate_sel}**\n\n"
                    f"** Week 1 — Immediate Action**\n\n"
                    "- Inspect all plants thoroughly; tag infected specimens.\n\n"
                    "- Remove and bag all visibly infected leaves/stems. Do NOT compost.\n\n"
                    "- Apply copper-based fungicide to all plants as a protective measure.\n\n"
                    f"** Week 2 — Active Treatment**\n\n"
                    "- Reapply fungicide if rain has occurred in the past 48 hours.\n\n"

```

```

"- Improve plant spacing for better air circulation.\n"
"- Switch entirely to drip irrigation or base-only watering.\n\n"
17 Week 3 — Monitoring & Support**\n"
"- Inspect daily for new symptoms; document any changes with photos.\n"
"- If spreading: upgrade to systemic fungicide (mancozeb or chlorothalonil).\n"
"- Apply balanced NPK fertiliser (10-10-10) to boost plant immunity.\n\n"
17 Week 4 — Recovery Assessment**\n"
"- Evaluate overall recovery — healthy new growth is a positive sign.\n"
"- Begin preventive spray schedule every 14 days going forward.\n"
"- Plan crop rotation for next season to break the disease cycle.\n\n"
17 Add a Gemini API key to `\.env` for a fully AI-personalised plan.*

```

)

```

st.markdown("### 📋 Your Care Plan")
st.success(care_plan)

```

```

dl1, dl2 = st.columns(2)
with dl1:

```

```

    st.download_button("📄 Download TXT", data=care_plan,

```

```

file_name=f"care_plan_{plant_sel}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.txt",
    mime="text/plain", use_container_width=True)

```

```

with dl2:

```

```

    care_json = json.dumps({
        "plant": plant_sel, "disease": disease_sel, "severity": sev_sel,
        "climate": climate_sel, "notes": extra, "plan": care_plan,
        "generated_at": datetime.now().isoformat(),
    }, indent=2)

```

```

    st.download_button("📄 Download JSON", data=care_json,

```

```

file_name=f"care_plan_{plant_sel}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json",
    mime="application/json", use_container_width=True)

```

— Tab 6: Encyclopedia

```

ENCYCLOPEDIA: list[dict] = [

```

```

    {

```

```

        "name": "Early Blight",      "pathogen": "Alternaria solani",      "emoji": "🍅",

```

```

        "plants": ["Tomato", "Potato"],

```

```

        "symptoms": "Dark brown concentric rings forming a 'target' pattern on older leaves.

```

```

        Yellowing tissue surrounds lesions.",

```

```

        "causes": "Fungal spores spread via wind and water splash. Favours warm (24–29°C),
        humid conditions.",

```

```

        "treatment": "Apply copper-based or chlorothalonil fungicide. Remove infected leaves.
        Improve air circulation.",

```

```

        "prevention": "3-year crop rotation, resistant varieties, drip irrigation.",

```

```

        "severity_risk": "Moderate", "spread_rate": "Moderate",

```

```

},
{
  "name": "Late Blight",      "pathogen": "Phytophthora infestans",   "emoji": "🍄",
  "plants": ["Tomato", "Potato"],
  "symptoms": "Water-soaked, dark brown–black lesions. White mould on leaf undersides. Rapid tissue collapse.",
  "causes": "Oomycete pathogen spread by airborne spores. Cool (10–25°C), wet conditions accelerate spread drastically.",
  "treatment": "Systemic fungicide (mancozeb, metalaxyl). Remove and destroy ALL infected material immediately.",
  "prevention": "Certified disease-free seed, resistant varieties, avoid overhead irrigation.",
  "severity_risk": "Very High — can destroy entire crop in days", "spread_rate": "Very Fast",
},
{
  "name": "Powdery Mildew",    "pathogen": "Erysiphe cichoracearum",   "emoji": "☁️",
  "plants": ["Apple", "Wheat", "Tomato"],
  "symptoms": "White powdery coating on leaf surfaces. Leaves curl, yellow, and may drop prematurely.",
  "causes": "Fungal spores spread by wind. Thrives in dry weather with moderate temperatures — unusual among fungi.",
  "treatment": "Neem oil, potassium bicarbonate, or sulfur-based fungicide. Prune dense foliage.",
  "prevention": "Plant resistant varieties, avoid excess nitrogen, ensure good air circulation.",
  "severity_risk": "Low to Moderate", "spread_rate": "Moderate",
},
{
  "name": "Septoria Leaf Spot", "pathogen": "Septoria lycopersici",   "emoji": "🔵",
  "plants": ["Tomato", "Wheat"],
  "symptoms": "Small circular spots with dark edges and light grey centres on lower leaves. Spreads upward progressively.",
  "causes": "Fungal spores in soil splashed onto lower leaves by rain or irrigation. Favours wet, warm weather.",
  "treatment": "Copper-based or chlorothalonil fungicide. Remove lower infected leaves immediately.",
  "prevention": "Mulch to prevent soil splash, stake plants for airflow, rotate crops annually.",
  "severity_risk": "Moderate", "spread_rate": "Moderate",
},
{
  "name": "Rust",                "pathogen": "Puccinia spp.",           "emoji": "🟡",
  "plants": ["Wheat", "Corn", "Apple"],
  "symptoms": "Orange, yellow, or rust-coloured pustules on both leaf surfaces and undersides. Leaves yellow and die.",
  "causes": "Obligate fungal parasite. Wind-dispersed spores. Favours moderate temperatures with high humidity.",

```

```

    "treatment": "Triazole or strobilurin fungicide. Remove severely infected plants.",
    "prevention": "Resistant cultivars, early fungicide applications, avoid dense planting.",
    "severity_risk": "High", "spread_rate": "Fast in windy, humid conditions",
  },
  {
    "name": "Gray Leaf Spot",    "pathogen": "Cercospora zeae-maydis",  "emoji": "❤️",
    "plants": ["Corn"],
    "symptoms": "Rectangular grey-to-tan lesions with parallel sides between leaf veins.
Leaves die from tips inward.",
    "causes": "Fungal. Favours high humidity, minimal wind, dense planting. Overwinters in
crop debris.",
    "treatment": "Strobilurin or triazole fungicide at early stages. Improve field drainage.",
    "prevention": "Crop rotation, deep tillage to bury debris, resistant hybrids, avoid late
planting.",
    "severity_risk": "Moderate to High", "spread_rate": "Moderate",
  },
  {
    "name": "Leaf Scab",          "pathogen": "Venturia inaequalis",    "emoji": "🟤",
    "plants": ["Apple"],
    "symptoms": "Olive-brown to black velvety spots on leaves and fruit. Infected leaves
may drop early.",
    "causes": "Fungal. Primary infection from overwintered spores in fallen leaves. Spreads
rapidly during spring rains.",
    "treatment": "Protective fungicide before rain events. Rake and destroy all fallen
leaves.",
    "prevention": "Resistant apple varieties, proper pruning, remove leaf litter, dormant
copper spray.",
    "severity_risk": "Moderate", "spread_rate": "Moderate in wet springs",
  },
  {
    "name": "Northern Corn Leaf Blight", "pathogen": "Exserohilum turcicum", "emoji": "🌽",
    "plants": ["Corn"],
    "symptoms": "Long (10–15 cm) cigar-shaped grey-green to tan lesions. Entire leaf may
die if untreated.",
    "causes": "Fungal spores overwinter in debris and spread by wind and rain. Favours
cool, humid weather.",
    "treatment": "Foliar fungicide at first sign of symptoms. Resistant hybrids are the most
effective defence.",
    "prevention": "Resistant corn hybrids, crop rotation, deep tillage to bury infected
residue.",
    "severity_risk": "High if early", "spread_rate": "Moderate to Fast",
  },
]

```

```

def tab_encyclopedia():
    st.markdown("## 📖 Disease Encyclopedia")
    st.markdown("Complete reference guide for all supported plant diseases.")

```

```

search      = st.text_input("🔍 Search diseases, pathogens, or plants...",
                           placeholder="e.g. fungal, tomato, rust...")
filter_plant_enc = st.selectbox("🌿 Filter by Plant", ["All"] + SUPPORTED_PLANTS)

```

```

filtered_enc = ENCYCLOPEDIA

```

```

if search:

```

```

    q = search.lower()
    filtered_enc = [
        e for e in filtered_enc
        if q in e["name"].lower() or q in e["pathogen"].lower()
        or q in e["symptoms"].lower()
        or any(q in p.lower() for p in e["plants"])
    ]

```

```

if filter_plant_enc != "All":

```

```

    filtered_enc = [e for e in filtered_enc if filter_plant_enc in e["plants"]]

```

```

if not filtered_enc:

```

```

    st.warning("No diseases match your search."); return

```

```

st.markdown(f"Showing **{len(filtered_enc)}** of {len(ENCYCLOPEDIA)} diseases.")

```

```

st.markdown("---")

```

```

risk_colour_map = {

```

```

    "Low to Moderate": "#f39c12",
    "Moderate":        "#e67e22",
    "Moderate to High": "#e74c3c",
    "High":            "#c0392b",
    "High if early":   "#e74c3c",
    "Very High — can destroy entire crop in days": "#7f0000",
}

```

```

for entry in filtered_enc:

```

```

    with st.expander(f'{entry["emoji"]} {entry["name"]} — *{entry["pathogen"]}*'):

```

```

        e1, e2 = st.columns([2, 1])

```

```

        with e1:

```

```

            st.markdown(f'***🌿 Affected Plants:** {', '.join(entry['plants'])}')

```

```

            st.markdown(f'***🦠 Pathogen:** *{entry["pathogen"]}*')

```

```

            st.markdown(f'***⚠️ Severity Risk:** {entry["severity_risk"]}')

```

```

            st.markdown(f'***📈 Spread Rate:** {entry["spread_rate"]}')

```

```

        with e2:

```

```

            rc = risk_colour_map.get(entry["severity_risk"], "#888")

```

```

            st.markdown(

```

```

                f'<div style="text-align:center;padding:18px;background:{rc}18;'

```

```

                f'border:2px solid {rc};border-radius:10px">'

```

```

                f'<div style="font-size:2rem">{entry["emoji"]}</div>'

```

```

                f'<div style="color:{rc};font-weight:700;font-size:0.85rem">'

```

```

                f'{entry["severity_risk"]}</div></div>',

```

```

        unsafe_allow_html=True,
    )
    st.markdown("---")
    st.markdown(f"📋 Symptoms: {entry['symptoms']}")
    st.markdown(f"🦠 Causes & Conditions: {entry['causes']}")
    tc, pc = st.columns(2)
    with tc:
        st.markdown(f"<div class='warning-card'><b>💊  
Treatment</b><br>{entry['treatment']}</div>",
                    unsafe_allow_html=True)
    with pc:
        st.markdown(f"<div class='disease-card'><b>🛡️  
Prevention</b><br>{entry['prevention']}</div>",
                    unsafe_allow_html=True)

```

— Tab 7: About

```

def tab_about():
    st.markdown("## 📄 About Leaf Health Check")

    a1, a2 = st.columns([2, 1])
    with a1:
        st.markdown(f"""
        ### 🎯 Mission
        Leaf Health Check is an AI-powered agricultural tool that helps farmers, gardeners,
        and agronomists rapidly diagnose plant leaf diseases, assess severity, and receive
        actionable rescue recommendations — all from a single leaf photograph.

        ### 🧠 Technology Stack
        | Layer | Technology |
        |---|---|
        | Frontend | Streamlit 1.28+ |
        | AI Models | TensorFlow / Keras — EfficientNetB0 CNN |
        | Computer Vision | OpenCV 4.8, Pillow 10 |
        | AI Assistant | Google Gemini Pro (optional) |
        | Database | SQLite 3 (built-in Python) |
        | Data Processing | Pandas, NumPy, Matplotlib |
        | Runtime | Python 3.12 |

        ### 🌱 Supported Scope
        - 5 plant species: Tomato, Potato, Apple, Corn, Wheat
        - 8 disease classes + Healthy detection
        - 5 severity grades: Healthy → Mild → Moderate → Severe → Dying
        - 90+ rescue tips stored in SQLite database
        - 8 encyclopedia entries with complete disease profiles
        """)

```


📊 Analysis Pipeline

...

Image Upload

- ↓ Validation & Preprocessing (OpenCV + PIL)
- ↓ HSV Pixel-level Discoloration Analysis
- ↓ CNN Disease Detection (EfficientNetB0)
- ↓ CNN Plant Species Classification
- ↓ Severity Grading (weighted score algorithm)
- ↓ Rescue Recommendation Lookup (SQLite + fallback)
- ↓ Health Score Computation (0–100)
- ↓ Heatmap Overlay Generation
- ↓ Interactive Charts (Matplotlib)
- ↓ Results + Export (JSON / CSV)

...

⚠️ Disclaimer

This application provides ****AI-assisted diagnosis for educational purposes****.
For critical agricultural or commercial decisions, always consult a qualified
plant pathologist or agricultural extension officer.

""")

with a2:

```
st.markdown("### 📈 App Statistics")
render_metric_card("App Version", APP_VERSION)
render_metric_card("Session Analyses", str(st.session_state.total_analyses))
render_metric_card("Session Start", st.session_state.session_start[:10])
render_metric_card("Encyclopedia Entries", str(len(ENCYCLOPEDIA)))
render_metric_card("Disease Classes", "8")
render_metric_card("Plant Species", "5")
render_metric_card("Severity Grades", "5")
render_metric_card("Rescue Tips in DB", "90+")
```

```
st.markdown("---")
```

```
st.markdown("""
```

🚀 Running for the First Time

```
```bash
```

```
1. Install dependencies (requires Python 3.12)
```

```
pip install -r requirements.txt
```

```
2. Generate AI model weight files (~86 MB total)
```

```
python model/create_models.py
```

```
3. Initialise the SQLite database
```

```
python database/init_db.py
```

```
4. Launch the Streamlit app
```

```
streamlit run app.py
```

```
```
```

...

- ...

Th

UUUUUU

 π

with

me

Г

1

11

"

"

1

```

        "📋 Care Plan",
        "📖 Encyclopedia",
        "ℹ️ About",
    ],
    label_visibility="collapsed",
)

st.markdown("---")
st.markdown("### 📌 Quick Guide")
st.markdown("""
1. Upload a leaf photo
2. Analyze — AI diagnosis
3. Review heatmap & charts
4. Follow rescue tips
5. Track trends in Dashboard
""")

st.markdown("---")
st.markdown("### ⚙️ System Status")
db_path = init_db()
ge = st.session_state.gemini_engine
st.success("✅ Database: Active") if db_path else st.error("❌ Database: Error")
if ge and ge._initialized:
    st.success("✅ Gemini AI: Connected")
else:
    st.warning("⚠️ Gemini AI: No key set")

if st.button("🔄 Clear Model Cache"):
    st.cache_resource.clear()
    st.session_state.model = None
    st.success("Cache cleared — models will reload on next analysis.")

st.markdown("---")
st.caption(f"v{APP_VERSION} · {st.session_state.total_analyses} analyses this session")

# Dispatch to selected tab
dispatch = {
    "🔍 Analyze Leaf": tab_analyze,
    "📈 Dashboard": tab_dashboard,
    "📊 History": tab_history,
    "🤖 AI Assistant": tab_ai_assistant,
    "📋 Care Plan": tab_care_plan,
    "📖 Encyclopedia": tab_encyclopedia,
    "ℹ️ About": tab_about,
}
dispatch.get(mode, tab_about)()

```

```
if __name__ == "__main__":  
    main()
```